Build plan

About 110 small steps in 14 milestones; each step is a few hours of work and ends with something you can run or test. From milestone 5 onward you can train with the app yourself, so every later step gets real use straight away.

M0 · Project setup

Done when: a blank app on your phone talks to a deployed API, and CI runs tests on every push.

Each step below says what to do, then gives checks to tick once you see the expected result. The stack choices for M0: pnpm workspaces, Fastify for the API, the native MongoDB driver with zod (no Mongoose), tsup to build the API, and Render to host it.

0 · Before you start

- Install Node.js LTS (22 or newer) and Git. Turn on pnpm with `corepack enable`.
- Install **Expo Go** on your phone.
- Create free accounts: GitHub, MongoDB Atlas, Render and Expo (sign in to Expo Go on your phone with it).

You should see:

- [ ] `node -v` prints v22 or higher
- [ ] `pnpm -v` prints a version number

1 · Create the monorepo

```
mkdir fit-coach && cd fit-coach
git init
pnpm init
corepack use pnpm@latest     # pins pnpm in package.json (CI reads this)
mkdir apps packages
```

Create `pnpm-workspace.yaml`:

```
packages:
  - "apps/*"
  - "packages/*"
```

Add `nodeLinker: hoisted` as a line in `pnpm-workspace.yaml` (React Native tooling expects a flat `node_modules`; pnpm 11 and later ignore this setting in `.npmrc`), and create a `.gitignore` containing `node_modules`, `dist`, `.expo`, `.env` and `.env.*`.

You should see:

- [ ] Root `package.json` has a `packageManager` field like `"pnpm@10.x"`
- [ ] `pnpm install` finishes with no errors

2 · Shared TypeScript, ESLint and Prettier

```
pnpm add -Dw typescript eslint @eslint/js typescript-eslint prettier
```

Create `tsconfig.base.json` at the root:

```
{
  "compilerOptions": {
    "strict": true,
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "esModuleInterop": true,
    "skipLibCheck": true,
    "noEmit": true
  }
}
```

Create `eslint.config.mjs`:

```
import js from "@eslint/js";
import tseslint from "typescript-eslint";
import { defineConfig, globalIgnores } from "eslint/config";

export default defineConfig(
  globalIgnores(["**/dist/**", "**/.expo/**"]),
  {
    files: ["**/*.{js,mjs,cjs,ts,tsx}"],
    extends: [js.configs.recommended, tseslint.configs.recommended],
  }
);
```

Add a `.prettierrc` (e.g. `{ "singleQuote": false, "printWidth": 100 }`), and these root scripts:

```
"scripts": {
  "lint": "eslint .",
  "format": "prettier --write .",
  "typecheck": "pnpm -r typecheck",
  "test": "pnpm -r test"
}
```

You should see:

- [ ] `pnpm lint` runs and reports no problems
- [ ] A throwaway file with an unused variable makes `pnpm lint` fail (then delete it)

3 · Expo app with a Hello screen

```
cd apps
pnpm dlx create-expo-app@latest mobile   # npx is blocked in a pnpm repo
pnpm --filter mobile exec expo login --browser   # once; --browser allows
GitHub sign-in
```

- In `apps/mobile/package.json`, rename the package to `@fit/mobile` and add `"typecheck": "tsc --noEmit"`. Leave its `tsconfig.json` as Expo made it.
- The default template uses expo-router. Replace the home screen's content with a single `<Text>Hello, Alec</Text>`.
- Run it from the repo root: `pnpm --filter @fit/mobile start`, then scan the QR code with your phone camera. Phone and laptop must be on the same Wi-Fi; if they can't connect, use `start --tunnel`.

You should see:

- [ ] "Hello, Alec" on your phone in Expo Go
- [ ] Changing the text and saving updates the phone within a couple of seconds

4 · Fastify API with `/health`

```
mkdir -p apps/api/src && cd apps/api
pnpm init
pnpm add fastify
pnpm add -D tsx tsup typescript @types/node
```

Set `"name": "@fit/api"` and `"type": "module"` in its `package.json`, a `tsconfig.json` that extends `../../tsconfig.base.json`, and `src/server.ts`:

```
import Fastify from "fastify";

const app = Fastify({ logger: true });

app.get("/health", async () => ({ status: "ok" }));

const port = Number(process.env.PORT ?? 3000);
await app.listen({ port, host: "0.0.0.0" });
```

Scripts:

```
"scripts": {
  "dev": "tsx watch --env-file=.env src/server.ts",
  "build": "tsup",
  "start": "node dist/server.js",
  "typecheck": "tsc --noEmit"
}
```

And `apps/api/tsup.config.ts`, next to the API's `package.json` (not in `src`). The `noExternal` line makes the build copy `@fit/shared` into `dist/server.js` from step 6 onwards:

```
import { defineConfig } from "tsup";

export default defineConfig({
  entry: ["src/server.ts"],
  format: ["esm"],
  target: "node22",
  clean: true,
  noExternal: ["@fit/shared"],
});
```

Create an empty `.env` for now, then run `pnpm --filter @fit/api dev`.

You should see:

- [ ] A log line saying the server is listening on port 3000
- [ ] `curl localhost:3000/health` returns `{"status":"ok"}`
- [ ] Your phone's browser opening `http://<laptop-ip>:3000/health` shows the same (this proves the phone can reach your laptop)
- [ ] `pnpm --filter @fit/api build` creates `dist/server.js`

5 · MongoDB Atlas

- In Atlas, create a free cluster, a database user with a strong password, and allow your current IP under Network Access.
- Copy the connection string into `apps/api/.env` as `MONGODB_URI=...`, with the database name `fitcoach` in the path.
- `pnpm add mongodb --filter @fit/api`, then add `src/db.ts`:

```
import { MongoClient } from "mongodb";

export const client = new MongoClient(process.env.MONGODB_URI!);
export const db = client.db();

export async function pingDb(): Promise<"up" | "down"> {
  try {
    await db.command({ ping: 1 });
    return "up";
  } catch {
    return "down";
  }
}
```

Change the health route to return `{ status: "ok", db: await pingDb() }`.

You should see:

- [ ] `/health` returns `{"status":"ok","db":"up"}`
- [ ] With a wrong password in `.env`, it returns `"db":"down"` and the API keeps running instead of crashing (then fix it)
- [ ] `git status` does not list `.env`

6 · Shared zod schemas

```
mkdir -p packages/shared/src && cd packages/shared
pnpm init
pnpm add zod
```

In its `package.json`: `"name": "@fit/shared"`, `"main": "src/index.ts"`, `"types": "src/index.ts"` and a `typecheck` script. Give it (and `@fit/engines` in step 7) a `tsconfig.json` that extends the base. No build step: tsx, tsup and Expo all read the TypeScript source directly. Add `src/index.ts`:

```
import { z } from "zod";

export const HealthResponse = z.object({
  status: z.literal("ok"),
  db: z.enum(["up", "down"]),
});
export type HealthResponse = z.infer<typeof HealthResponse>;
```

- Add it to both apps: `pnpm add "@fit/shared@workspace:*" --filter @fit/api --filter @fit/mobile`.
- In the API, return `HealthResponse.parse({ ... })` from `/health`.
- In `apps/mobile/.env`, set `EXPO_PUBLIC_API_URL=http://<laptop-ip>:3000`. On the Hello screen, fetch `/health`, parse it with `HealthResponse`, and show `API ok · DB up` (or the error).

You should see:

- [ ] Your phone shows "API ok · DB up"
- [ ] Renaming `db` in the shared schema makes `pnpm typecheck` fail in both apps (then undo it)
- [ ] Stopping the API makes the phone show an error rather than crash

7 · Vitest in the engines package

```
mkdir -p packages/engines/src && cd packages/engines
pnpm init
pnpm add -D vitest
```

Edit the `package.json` that `pnpm init` created so it looks like this (pnpm adds the `devDependencies` entry for you):

```
{
  "name": "@fit/engines",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "main": "src/index.ts",
  "scripts": {
    "test": "vitest run",
    "typecheck": "tsc --noEmit"
  },
  "devDependencies": {
    "vitest": "^3.x.x"
  }
}
```

Add `tsconfig.json` with `{ "extends": "../../tsconfig.base.json", "include": ["src"] }`, and `src/index.ts` with `export * from "./e1rm";`.

Make the first test a real engine function rather than a dummy: the Epley one-rep-max estimate you'll need for the baseline test in M4.

```
// src/e1rm.ts
export function e1rm(loadKg: number, reps: number): number {
  if (reps <= 0) throw new Error("reps must be positive");
  return loadKg * (1 + reps / 30);
}

// src/e1rm.test.ts
import { describe, expect, it } from "vitest";
import { e1rm } from "./e1rm";

describe("e1rm", () => {
  it("estimates 100 kg x 5 as about 116.7 kg", () => {
    expect(e1rm(100, 5)).toBeCloseTo(116.67, 1);
  });
  it("rejects zero reps", () => {
    expect(() => e1rm(100, 0)).toThrow();
  });
});
```

You should see:

- [ ] `pnpm test` from the root prints 2 passed
- [ ] Changing 116.67 to 120 turns the test red (then undo it)

8 · GitHub and CI

- Create an empty GitHub repo, then commit and push.
- Add `.github/workflows/ci.yml` (use the latest major versions of these actions if newer ones exist):

```
name: CI
on: [push, pull_request]

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: pnpm
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint
      - run: pnpm typecheck
      - run: pnpm test
```

You should see:

- [ ] A green tick next to your latest commit on GitHub, with the lint, typecheck and test steps all passing
- [ ] Pushing a deliberately broken test gives a red cross (then fix and push again)

9 · Deploy the API and point the app at it

- In Render, create a new Web Service from your GitHub repo:
  - Build command: `corepack enable && pnpm install --frozen-lockfile && pnpm --filter @fit/api build`
  - Start command: `pnpm --filter @fit/api start`
  - Environment variable: `MONGODB_URI` (same value as your `.env`)
- In Atlas Network Access, allow `0.0.0.0/0`. Render's free tier has no fixed IP address, so the strong database password is what protects you.
- In `apps/mobile/package.json`, add a script that runs the app against the deployed API: `"start:prod-api": "EXPO_PUBLIC_API_URL=https://<your-service>.onrender.com expo start --clear"` (macOS/Linux syntax).

If Render's free plan has changed, Railway and Fly.io follow the same pattern: build command, start command, one environment variable.

You should see:

- [ ] `https://<your-service>.onrender.com/health` in a browser returns `{"status":"ok","db":"up"}`. The first request after idling can take up to a minute while the free instance wakes.
- [ ] With `start:prod-api` and the local API stopped (Ctrl+C), the phone still shows "API ok · DB up"
- [ ] Render's logs show that request
- [ ] Every push to `main` triggers a new deploy on Render

M1 · Auth and profile

Done when: you can sign up, fill in your details, and see them saved after restarting the app.

Stack choices for M1: Clerk for sign-in, `@clerk/fastify` to check tokens on the API, React Hook Form with zod for the forms, and the user's Clerk ID (not email) as the key in MongoDB, since Clerk already stores the email. Clerk's SDKs change often, so follow its Expo quickstart for exact function names; the flow below stays the same.

1 · Set up Clerk

- Create an application at clerk.com. Turn on **Email** with **verification code** (no password), and optionally **GitHub**.
- Copy the two keys from **API Keys**:
  - `apps/mobile/.env`: `EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test_...`
  - `apps/api/.env`: `CLERK_PUBLISHABLE_KEY=pk_test_...` and `CLERK_SECRET_KEY=sk_test_...`
- The secret key must never go in the mobile app: anything starting `EXPO_PUBLIC_` is readable inside the app.

You should see:

- [ ] Your app in the Clerk dashboard with 0 users
- [ ] Both `.env` files listed nowhere in `git status`

2 · Add Clerk to the Expo app

Install with `expo install`, which picks versions that match your SDK:

```
pnpm --filter @fit/mobile exec expo install @clerk/expo expo-secure-store
```

Wrap the whole app in `ClerkProvider` in `app/_layout.tsx`. The token cache stores the session in the phone's secure storage, so you stay signed in after closing the app:

```
import { ClerkProvider } from "@clerk/expo";
import { tokenCache } from "@clerk/expo/token-cache";
import { Slot } from "expo-router";

const publishableKey = process.env.EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY!;

export default function RootLayout() {
  return (
    <ClerkProvider publishableKey={publishableKey} tokenCache={tokenCache}>
      <Slot />
    </ClerkProvider>
  );
}
```

You should see:

- [ ] The app still loads on your phone with no red error screen

3 · Sign-up, sign-in and route guards

Organise screens into three expo-router groups (folders in brackets don't appear in the URL):

```
app/
├── _layout.tsx            ClerkProvider (step 2)
├── (auth)/                sign-in.tsx, sign-up.tsx
├── (onboarding)/          body.tsx, goal.tsx, injuries.tsx
└── (app)/                 index.tsx (home), profile.tsx
```

Move or delete the template's example tabs. There are no Clerk screen templates to copy; the simplest start is Clerk's **hosted authentication**, which opens Clerk's own sign-in pages in a browser sheet and works in Expo Go. Enable the **Native API** on the Native applications page in the Clerk dashboard, install the extra packages (`expo install expo-auth-session expo-crypto expo-web-browser`), and make `(auth)/sign-in.tsx` two buttons that call `startHostedAuth({ mode: "sign-in" })` and `startHostedAuth({ mode: "sign-up" })` from `useHostedAuth` (`@clerk/expo/hosted-auth`). A fully custom in-app form (Clerk's custom-flow guides) can replace it later without touching anything else.

Each group's `_layout.tsx` decides who may enter. The app group sends signed-out users away:

```
// app/(app)/_layout.tsx
import { useAuth } from "@clerk/expo";
import { Redirect, Stack } from "expo-router";

export default function AppLayout() {
  const { isLoaded, isSignedIn } = useAuth();
  if (!isLoaded) return null;
  if (!isSignedIn) return <Redirect href="/sign-in" />;
  return <Stack />;
}
```

The `(auth)` layout does the reverse: signed-in users are redirected to `/`. Add a **Sign out** button on the home screen that calls `signOut()` from `useAuth`.

You should see:

- [ ] Signing up with your real email sends a 6-digit code, and entering it lands you on the home screen
- [ ] You appear as a user in the Clerk dashboard
- [ ] After fully closing Expo Go and reopening, you're still signed in
- [ ] Sign out returns you to the sign-in screen, and the back gesture can't get you into the app

4 · Protect API routes

```
pnpm --filter @fit/api add @clerk/fastify
```

Register the plugin (it reads the two Clerk keys from `.env`) and add a guard you can attach to any route:

```
import { clerkPlugin, getAuth } from "@clerk/fastify";
import type { FastifyReply, FastifyRequest } from "fastify";

await app.register(clerkPlugin);

export async function requireAuth(req: FastifyRequest, reply: FastifyReply) {
  const { userId } = getAuth(req);
  if (!userId) return reply.code(401).send({ error: "unauthorized" });
}

app.get("/me", { preHandler: requireAuth }, async (req) => ({
  clerkUserId: getAuth(req).userId,
}));
```

`/health` stays public because it has no `preHandler`.

You should see:

- [ ] `curl localhost:3000/me` returns 401 `{"error":"unauthorized"}`
- [ ] `curl localhost:3000/health` still returns `{"status":"ok","db":"up"}`

5 · Call the API with your token

Add `apps/mobile/lib/api.ts`, a hook that attaches the Clerk token to every request:

```
import { useAuth } from "@clerk/expo";

const API_URL = process.env.EXPO_PUBLIC_API_URL;

export function useApi() {
  const { getToken } = useAuth();

  return async (path: string, init: RequestInit = {}): Promise<unknown> => {
    const token = await getToken();
    const res = await fetch(`${API_URL}${path}`, {
      ...init,
      headers: {
        "Content-Type": "application/json",
        ...(token ? { Authorization: `Bearer ${token}` } : {}),
      },
    });
    if (!res.ok) throw new Error(`${res.status}: ${await res.text()}`);
    return res.json();
  };
}
```

It returns `unknown` on purpose: every caller parses the result with a zod schema, so bad data fails loudly. On the home screen, call `/me` and show the ID.

You should see:

- [ ] The home screen shows "Signed in as user_..."
- [ ] The API terminal logs a 200 for `GET /me`

6 · Create the user document on first sign-in

In `packages/shared`, add the profile schemas. Every field is required for a finished profile; `.partial()` covers half-done onboarding:

```
export const Goal = z.enum(["lose_fat", "build_muscle", "recomp",
"athletic"]);
export const InjuryTag = z.enum([
  "neck", "shoulder", "elbow", "wrist", "lower_back", "hip", "knee", "ankle",
]);

export const UserProfile = z.object({
  sex: z.enum(["male", "female"]),          // for calorie estimates
  birthYear: z.number().int().min(1930).max(new Date().getFullYear() - 16),
  heightCm: z.number().min(120).max(230),
  weightKg: z.number().min(35).max(250),
  goal: Goal,
  daysPerWeek: z.number().int().min(2).max(6),
  injuries: z.array(InjuryTag),
});
export type UserProfile = z.infer<typeof UserProfile>;

export const Me = z.object({
  id: z.string(),
  profile: UserProfile.partial(),
  onboardedAt: z.iso.datetime().nullable(),
});
```

In the API, add `src/users.ts`. The upsert creates the document the first time and just returns it every time after:

```
export const users = db.collection("users");

export async function ensureUserIndexes() {
  await users.createIndex({ clerkUserId: 1 }, { unique: true });
}

export async function findOrCreateUser(clerkUserId: string) {
  return users.findOneAndUpdate(
    { clerkUserId },
    { $setOnInsert: { clerkUserId, createdAt: new Date(), profile: {},
onboardedAt: null } },
    { upsert: true, returnDocument: "after" }
  );
}
```

Call `ensureUserIndexes()` once at startup, and change `/me` to return the user in the `Me` shape.

You should see:

- [ ] In Atlas → Collections, a `fitcoach` database with a `users` collection holding one document
- [ ] Signing out and back in does not create a second document
- [ ] The Indexes tab shows a unique index on `clerkUserId`

7 · Save profile fields, validated by zod

```
pnpm --filter @fit/api add fastify-type-provider-zod
```

```
import { serializerCompiler, validatorCompiler, type ZodTypeProvider } from
"fastify-type-provider-zod";

app.setValidatorCompiler(validatorCompiler);
app.setSerializerCompiler(serializerCompiler);

app.withTypeProvider<ZodTypeProvider>().patch(
  "/me/profile",
  { preHandler: requireAuth, schema: { body: UserProfile.partial() } },
  async (req) => {
    const set = Object.fromEntries(
      Object.entries(req.body).map(([key, value]) => [`profile.${key}`,
value])
    );
    await users.updateOne({ clerkUserId: getAuth(req).userId }, { $set: set
});
    return { ok: true };
  }
);
```

`req.body` is fully typed from the schema. The `profile.` prefix updates only the fields sent, leaving the rest of the profile alone. Add `POST /me/onboarding/complete` too: it checks the saved profile with the full `UserProfile.safeParse`, sets `onboardedAt` if it passes, and returns 400 if anything is missing.

You should see:

Turn on zod validation in `server.ts`, and register the `/me` routes from their own file (delete the old `/me` route from `server.ts`):

```
import { serializerCompiler, validatorCompiler } from "fastify-type-provider-
zod";
import { meRoutes } from "./routes/me";

app.setValidatorCompiler(validatorCompiler);   // checks incoming requests
app.setSerializerCompiler(serializerCompiler); // checks outgoing responses

await app.register(clerkPlugin);
await ensureUserIndexes();
await app.register(meRoutes);
```

Create `src/routes/me.ts`. It's a Fastify plugin, so its `preHandler` hook protects only the routes in this file, and `/health` stays public:

```
import { getAuth } from "@clerk/fastify";
import type { FastifyPluginAsyncZod } from "fastify-type-provider-zod";
import { UserProfile } from "@fit/shared";
import { requireAuth } from "../auth";
import { findOrCreateUser, toMe, users } from "../users";

export const meRoutes: FastifyPluginAsyncZod = async (app) => {
  app.addHook("preHandler", requireAuth);

  app.get("/me", async (req) => {
    const user = await findOrCreateUser(getAuth(req).userId!);
    return toMe(user);
  });

  app.patch("/me/profile", { schema: { body: UserProfile.partial() } }, async
(req) => {
    const user = await users.findOneAndUpdate(
      { clerkUserId: getAuth(req).userId! },
      [{ $set: { profile: { $mergeObjects: ["$profile", req.body] } } }],
      { returnDocument: "after" }
    );
    if (!user) throw new Error("User not found");
    return toMe(user);
  });

  app.post("/me/onboarding/complete", async (req, reply) => {
    const user = await findOrCreateUser(getAuth(req).userId!);
    const check = UserProfile.safeParse(user.profile);
    if (!check.success) {
      return reply.code(400).send({
        error: "profile incomplete",
        missing: check.error.issues.map((issue) => issue.path.join(".")),
      });
    }
    const updated = await users.findOneAndUpdate(
      { _id: user._id },
      { $set: { onboardedAt: user.onboardedAt ?? new Date() } },
      { returnDocument: "after" }
    );
    return toMe(updated!);
  });
};
```

`req.body` is typed from the schema, and unknown fields are stripped. `$mergeObjects` merges the sent fields into the existing profile in one atomic update, leaving the rest alone. The completion route returns 400 with the list of missing fields until the profile passes the full `UserProfile` schema. Test from temporary buttons on the home screen, since a `curl` would need a short-lived Clerk token.

You should see:

- [ ] Sending `{ "heightCm": "tall" }` from the app returns a 400 that names the bad field
- [ ] Sending `{ "heightCm": 180 }`, then `{ "weightKg": 80 }`, leaves both saved in Atlas
- [ ] `POST /me/onboarding/complete` returns 400 listing the missing fields
- [ ] `/health` still works without signing in

8 · Onboarding screen: your body

Build `(onboarding)/body.tsx` with React Hook Form and the shared schema, the same pattern as yup's resolver:

```
const BodyForm = UserProfile.pick({ sex: true, birthYear: true, heightCm:
true, weightKg: true });
const form = useForm({ resolver: zodResolver(BodyForm) });
```

- Text inputs give strings, so convert in each `Controller`: `onChangeText={(t) => field.onChange(t === "" ? undefined : Number(t))}`, with `keyboardType="numeric"`.
- Sex is two big buttons, not a dropdown.
- **Next** sends the four fields to `PATCH /me/profile`, then goes to the goal screen.
- Store everything in metric; a units toggle for feet and pounds can come later as a display setting.

You should see:

The first onboarding screen asks for sex, birth year, height and weight, validates them with the shared schema as you type, and saves them through `PATCH /me/profile`. It's the template for every form that follows, so it's worth getting the pieces right here.

8a · Install the form libraries

```
pnpm --filter @fit/mobile add react-hook-form @hookform/resolvers
```

React Hook Form tracks field values and errors; `@hookform/resolvers` lets it validate with zod, exactly like the yup resolver you've used before.

8b · Give the shared schema friendly error messages

Zod's default messages ("Too big: expected number to be <=230") aren't for end users. Add messages in `packages/shared/src/index.ts`, so the API and the app both use them:

```
export const UserProfile = z.object({
  sex: z.enum(["male", "female"], { error: "Choose one" }),
  birthYear: z
    .number({ error: "Enter your birth year" })
    .int("Use a whole year, e.g. 1988")
    .min(1930, "Check your birth year")
    .max(new Date().getFullYear() - 16, "You must be at least 16"),
  heightCm: z
    .number({ error: "Enter your height" })
    .min(120, "Height must be at least 120 cm")
    .max(230, "Height must be at most 230 cm"),
  weightKg: z
    .number({ error: "Enter your weight" })
    .min(35, "Weight must be at least 35 kg")
    .max(250, "Weight must be at most 250 kg"),
  goal: Goal,
  daysPerWeek: z.number().int().min(2).max(6),
  injuries: z.array(InjuryTag),
});
```

8c · Protect the onboarding group: `src/app/(onboarding)/_layout.tsx`

```
import { useAuth } from "@clerk/expo";
import { Redirect, Stack } from "expo-router";

export default function OnboardingLayout() {
  const { isLoaded, isSignedIn } = useAuth();
  if (!isLoaded) return null;
  if (!isSignedIn) return <Redirect href="/sign-in" />;
  return <Stack screenOptions={{ title: "Set up", headerBackTitle: "Back" }}
/>;
}
```

The `Stack` gives the three onboarding screens a header with a back button, so you can step back from Goal to Body.

8d · A reusable number input: `src/components/NumberField.tsx`

Number inputs are fiddly: the text box holds a string, the form wants a number, and typing "70." on the way to "70.5" must not be turned into 70 and lose the dot. This component keeps the typed text separately and passes a number up:

```
import { useEffect, useState } from "react";
import { StyleSheet, Text, TextInput, View } from "react-native";

type Props = {
  label: string;
  unit?: string;
  value: number | undefined;
  onChange: (value: number | undefined) => void;
  onBlur: () => void;
  error?: string;
  decimals?: boolean;
};

export function NumberField({ label, unit, value, onChange, onBlur, error,
decimals }: Props) {
  const [text, setText] = useState(value === undefined ? "" : String(value));

  // When the form is filled from saved data, show it in the box
  useEffect(() => {
    if (value !== undefined && Number(text) !== value)
setText(String(value));
  }, [value]);

  return (
    <View style={styles.field}>
      <Text style={styles.label}>{label}</Text>
      <View style={styles.row}>
        <TextInput
          style={[styles.input, error && styles.inputError]}
          value={text}
          keyboardType={decimals ? "decimal-pad" : "number-pad"}
          onBlur={onBlur}
          onChangeText={(raw) => {
            const cleaned = raw.replace(",", "."); // some keyboards type
commas
            setText(cleaned);
            onChange(cleaned === "" ? undefined : Number(cleaned));
          }}
        />
        {unit && <Text style={styles.unit}>{unit}</Text>}
      </View>
      {error && <Text style={styles.error}>{error}</Text>}
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  field: { gap: 6 },
  label: { fontSize: 15, fontWeight: "600" },
  row: { flexDirection: "row", alignItems: "center", gap: 8 },
  input: { flex: 1, borderWidth: 1, borderColor: "#ccc", borderRadius: 8,
padding: 12, fontSize: 18 },
  inputError: { borderColor: "crimson" },
  unit: { fontSize: 16, color: "#666" },
  error: { color: "crimson" },
});
```

8e · The screen: `src/app/(onboarding)/body.tsx`

```
import { zodResolver } from "@hookform/resolvers/zod";
import { Me, UserProfile } from "@fit/shared";
import { useRouter } from "expo-router";
import { useEffect, useState } from "react";
import { Controller, useForm } from "react-hook-form";
import { ActivityIndicator, Button, Pressable, ScrollView, StyleSheet, Text,
View } from "react-native";
import { NumberField } from "../../components/NumberField";
import { useApi } from "../../lib/api";

const BodyForm = UserProfile.pick({ sex: true, birthYear: true, heightCm:
true, weightKg: true });

export default function BodyScreen() {
  const api = useApi();
  const router = useRouter();
  const [loading, setLoading] = useState(true);
  const [serverError, setServerError] = useState<string | null>(null);

  const { control, handleSubmit, reset, formState } = useForm({
    resolver: zodResolver(BodyForm),
    mode: "onBlur", // show errors when you leave a field, not on every
keystroke
  });

  // Pre-fill anything already saved, so a half-finished onboarding resumes
  useEffect(() => {
    api("/me")
      .then((json) => {
        const { profile } = Me.parse(json);
        reset({
          sex: profile.sex,
          birthYear: profile.birthYear,
```

```
          heightCm: profile.heightCm,
          weightKg: profile.weightKg,
        });
      })
      .catch((err: Error) => setServerError(err.message))
      .finally(() => setLoading(false));
  }, []);

  const onSubmit = handleSubmit(async (values) => {
    setServerError(null);
    try {
      await api("/me/profile", { method: "PATCH", body:
JSON.stringify(values) });
      router.push("/goal");
    } catch (err) {
      setServerError(err instanceof Error ? err.message : "Could not save");
    }
  });

  if (loading) return <ActivityIndicator style={{ flex: 1 }} />;

  return (
    <ScrollView contentContainerStyle={styles.container}
keyboardShouldPersistTaps="handled">
      <Text style={styles.title}>About you</Text>
      <Text style={styles.hint}>Used to estimate your calorie needs.</Text>

      <Controller
        control={control}
        name="sex"
        render={({ field, fieldState }) => (
          <View style={styles.field}>
            <Text style={styles.label}>Sex</Text>
            <View style={styles.row}>
              {(["male", "female"] as const).map((option) => (
                <Pressable
                  key={option}
                  onPress={() => field.onChange(option)}
                  style={[styles.choice, field.value === option &&
styles.choiceSelected]}
                >
                  <Text style={field.value === option &&
styles.choiceTextSelected}>
                    {option === "male" ? "Male" : "Female"}
                  </Text>
                </Pressable>
```

```
              ))}
            </View>
            {fieldState.error && <Text style={styles.error}>
{fieldState.error.message}</Text>}
          </View>
        )}
      />

      <Controller
        control={control}
        name="birthYear"
        render={({ field, fieldState }) => (
          <NumberField label="Birth year" {...field} error=
{fieldState.error?.message} />
        )}
      />
      <Controller
        control={control}
        name="heightCm"
        render={({ field, fieldState }) => (
          <NumberField label="Height" unit="cm" decimals {...field} error=
{fieldState.error?.message} />
        )}
      />
      <Controller
        control={control}
        name="weightKg"
        render={({ field, fieldState }) => (
          <NumberField label="Weight" unit="kg" decimals {...field} error=
{fieldState.error?.message} />
        )}
      />

      {serverError && <Text style={styles.error}>{serverError}</Text>}
      <Button title={formState.isSubmitting ? "Saving…" : "Next"} onPress=
{onSubmit} disabled={formState.isSubmitting} />
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  container: { padding: 20, gap: 20 },
  title: { fontSize: 26, fontWeight: "600" },
  hint: { color: "#666", marginTop: -12 },
  field: { gap: 6 },
  label: { fontSize: 15, fontWeight: "600" },
```

```
  row: { flexDirection: "row", gap: 12 },
  choice: { flex: 1, padding: 16, borderWidth: 1, borderColor: "#ccc",
borderRadius: 8, alignItems: "center" },
  choiceSelected: { backgroundColor: "#222", borderColor: "#222" },
  choiceTextSelected: { color: "white", fontWeight: "600" },
  error: { color: "crimson" },
});
```

How the pieces fit:

- **`UserProfile.pick({...})`** makes a schema with only this screen's four fields, all required. The same schema guards the API, so the app and server can't disagree about what's valid.
- **`zodResolver(BodyForm)`** runs that schema when you leave a field (`mode: "onBlur"`) and when you press Next; errors appear under each field.
- **`Controller`** connects React Native inputs, which aren't HTML elements, to React Hook Form. `{...field}` passes `value`, `onChange` and `onBlur` straight into `NumberField`.
- **`reset(...)`** fills the form from the saved profile, so closing the app halfway and coming back keeps your answers.
- **`handleSubmit`** only calls the function once validation passes, then the values go to `PATCH /me/profile` and the app moves to the goal screen. Until step 9 exists, `router.push("/goal")` will show a "not found" screen, which is fine for now.
- **Everything is stored in metric.** A feet-and-pounds display option can come later as a setting; the database never changes.

To reach this screen before step 10's redirect exists, temporarily add a button on the home screen: `<Button title="Onboarding" onPress={() => router.push("/body")} />`.

You should see:

- [ ] Pressing Next with empty fields shows four messages, e.g. "Enter your height"
- [ ] Height 300 shows "Height must be at most 230 cm", and Next doesn't send anything
- [ ] You can type 72.5 for weight without the decimal point disappearing
- [ ] Valid values save: Atlas shows them under `profile`, and the app moves on
- [ ] Going back to this screen shows your saved values already filled in
- [ ] `pnpm typecheck` and `pnpm lint` pass

9 · Onboarding screen: goal and schedule

`(onboarding)/goal.tsx`: four large tappable cards for the goal (Lose fat · Build muscle · Recomp · Athletic), each with a one-line description, plus a 2–6 row of buttons for days per week.

You should see:

- [ ] Next stays disabled until both are chosen
- [ ] Both values saved in Atlas

10 · Onboarding screen: injuries, then finish

`(onboarding)/injuries.tsx`: a grid of chips, one per `InjuryTag`, plus a **None** chip that clears the others. **Finish** saves the injuries, calls `POST /me/onboarding/complete`, then goes home.

Now extend the `(app)` guard from step 3: after the sign-in check, fetch `/me` and redirect to `/body` while `onboardedAt` is null. The onboarding screens pre-fill any saved values, so a half-finished onboarding resumes where it left off.

You should see:

- [ ] Finishing lands on the home screen and `onboardedAt` has a date in Atlas
- [ ] Closing the app halfway through onboarding and reopening it returns you to onboarding, with your earlier answers filled in
- [ ] Selecting None clears any other chips

11 · Profile screen

`(app)/profile.tsx` shows every profile value with an **Edit** button per section. Reuse the three onboarding forms as components, each saving through `PATCH /me/profile`. Move Sign out here from the home screen.

You should see:

- [ ] Changing your weight updates Atlas and still shows after restarting the app
- [ ] An invalid edit shows the same inline errors as onboarding
- [ ] `pnpm typecheck` and `pnpm lint` pass, and CI is green

M2 · Equipment and exercise library

Done when: the app lists only the exercises your home kit supports.

Design choices for M2: equipment profiles live inside the user document; the exercise library is plain TypeScript data in `@fit/engines` (so the planner's tests can use it) and is copied into MongoDB by a seed script; and the same `isAvailable()` function runs in the API and the app. Onboarding grows three screens after Injuries, so **Finish** (the call to `/me/onboarding/complete`) moves from the injuries screen to the end of this flow.

1 · Equipment schemas in `@fit/shared`

Add to `packages/shared/src/index.ts`:

```
export const EquipmentTag = z.enum([
  // Free weights
  "dumbbells", "kettlebells", "barbell", "plates", "ez_bar", "weight_vest",
"medicine_ball", "slam_ball",
  // Benches and racks
  "flat_bench", "adjustable_bench", "squat_rack", "power_rack", "landmine",
  // Bodyweight stations
  "pull_up_bar", "rings", "dip_station", "parallettes", "suspension_trainer",
"plyo_box", "ab_wheel", "sliders",
  // Bands and cables
  "loop_bands", "mini_bands", "tube_bands", "cable_machine",
  // Cardio and recovery
  "jump_rope", "stationary_bike", "rower", "treadmill", "air_bike",
"foam_roller",
]);
export type EquipmentTag = z.infer<typeof EquipmentTag>;

const kg = z.number().positive().max(200);

export const DumbbellLoads = z.discriminatedUnion("kind", [
  z.object({ kind: z.literal("adjustable"), minKg: kg, maxKg: kg, stepKg: kg
}),
  z.object({ kind: z.literal("fixed"), weightsKg: z.array(kg).min(1) }),
]);

export const BandLevel = z.enum(["light", "medium", "heavy", "extra_heavy"]);

export const EquipmentProfile = z.object({
  id: z.string(),
  name: z.string().min(1).max(40),
  location: z.enum(["gym", "home", "outdoors"]),
  isPrimary: z.boolean(),
  tags: z.array(EquipmentTag),
  loads: z
    .object({
      dumbbells: DumbbellLoads.optional(),
```

```
      kettlebells: z.object({ weightsKg: z.array(kg).min(1) }).optional(),
      loop_bands: z.object({ levels: z.array(BandLevel).min(1) }).optional(),
    })
    .default({}),
});
export type EquipmentProfile = z.infer<typeof EquipmentProfile>;
```

Also add `equipmentProfiles: z.array(EquipmentProfile)` to the `Me` schema, and a `PRESETS` list and an `EQUIPMENT_LABELS` map (tag → "Pull-up bar") for the screens:

```
export const PRESETS: { name: string; tags: EquipmentTag[] }[] = [
  { name: "Bodyweight only", tags: [] },
  { name: "Minimal: bar + bands", tags: ["pull_up_bar", "loop_bands"] },
  { name: "Dumbbells + bench", tags: ["dumbbells", "adjustable_bench"] },
  { name: "Kettlebells", tags: ["kettlebells"] },
  { name: "Full home gym", tags: ["barbell", "plates", "power_rack",
"adjustable_bench", "dumbbells", "pull_up_bar"] },
];
```

A **discriminated union** is a zod schema that picks its shape from one field: `kind: "adjustable"` requires min, max and step; `kind: "fixed"` requires a list. A commercial gym profile is every tag (`EquipmentTag.options`) with no `loads` , meaning "no weight limits".

You should see:

- [ ] `pnpm typecheck` passes
- [ ] In a scratch test, `DumbbellLoads.safeParse({ kind: "fixed" })` fails and names `weightsKg`

2 · Store profiles on the user (API)

- In `users.ts` , add `equipmentProfiles: EquipmentProfile[]` to `UserDoc` , `equipmentProfiles: []` to the `$setOnInsert` , and `equipmentProfiles: user.equipmentProfiles ?? []` to `toMe` (the `?? []` covers documents created before this field existed).
- In `routes/me.ts` , add two routes:

```
// Replace the whole list (location screen)
app.put(
  "/me/equipment-profiles",
  {
    schema: {
      body: z
        .array(EquipmentProfile.omit({ id: true }))
```

```
        .min(1)
        .refine((list) => list.filter((p) => p.isPrimary).length === 1,
"Exactly one profile must be primary"),
    },
  },
  async (req) => {
    const profiles = req.body.map((p) => ({ ...p, id: new
ObjectId().toHexString() }));
    const user = await users.findOneAndUpdate(
      { clerkUserId: getAuth(req).userId! },
      { $set: { equipmentProfiles: profiles } },
      { returnDocument: "after" }
    );
    return toMe(user!);
  }
);

// Update one profile's tags or loads (equipment and weights screens)
app.patch(
  "/me/equipment-profiles/:id",
  {
    schema: {
      params: z.object({ id: z.string() }),
      body: EquipmentProfile.pick({ tags: true, loads: true, name: true
}).partial(),
    },
  },
  async (req, reply) => {
    const set = Object.fromEntries(
      Object.entries(req.body).map(([key, value]) =>
[`equipmentProfiles.$.${key}`, value])
    );
    const user = await users.findOneAndUpdate(
      { clerkUserId: getAuth(req).userId!, "equipmentProfiles.id":
req.params.id },
      { $set: set },
      { returnDocument: "after" }
    );
    if (!user) return reply.code(404).send({ error: "profile not found" });
    return toMe(user);
  }
);
```

The server creates each profile's `id`, so the app never has to. In the PATCH, `equipmentProfiles.$` is MongoDB's **positional operator**: "the array item that matched `equipmentProfiles.id` in the filter". Finally, make `/me/onboarding/complete` also return

400 when `equipmentProfiles` is empty.

You should see:

- [ ] Sending two profiles both marked primary returns 400 with "Exactly one profile must be primary"
- [ ] A valid PUT shows the profiles in Atlas, each with an `id`
- [ ] PATCHing one profile's tags leaves the other profile untouched

3 · Onboarding screen: where do you train?

`src/app/(onboarding)/location.tsx`: four large cards. Each choice calls `PUT /me/equipment-profiles` straight away, then moves on:

Choice	Profiles created	Next screen
Commercial gym	Gym (primary, every tag)	Finish
Home	Home (primary, no tags yet)	Equipment
Outdoors / bodyweight only	Outdoors (primary, no tags)	Finish
A mix	Home (primary) + Gym	Equipment (for the home profile)

Change the injuries screen's button from Finish to **Next** → `/location`. "Finish" here means: call `POST /me/onboarding/complete`, then `router.replace("/")`. You can change which profile is primary later on the profile screen.

You should see:

- [ ] Choosing Commercial gym completes onboarding and lands on home
- [ ] Choosing A mix creates two profiles in Atlas, with Home as primary

4 · Onboarding screen: home equipment checklist

`src/app/(onboarding)/equipment.tsx`:

- Fetch `/me` and find the home profile; keep its `tags` in `useState` as the starting selection.
- **Presets** as a row of chips at the top. Tapping one replaces the selection with that preset's tags (it's a starting point; the user can then add or remove).

- **The checklist** grouped by category (use the comment groups from step 1), each tag a toggle chip labelled from `EQUIPMENT_LABELS`. A `Set<EquipmentTag>` makes toggling simple: `next.has(tag) ? next.delete(tag) : next.add(tag)`.
- **Next** sends `PATCH /me/equipment-profiles/:id` with `{ tags }`, then goes to the weights screen if any of `dumbbells`, `kettlebells` or `loop_bands` is selected, otherwise Finish.

You should see:

- [ ] Tapping "Dumbbells + bench" selects exactly those two chips
- [ ] With nothing loadable selected, Next skips the weights screen
- [ ] Going back and forward keeps your selection (it was saved)

5 · Onboarding screen: weights on hand

`src/app/(onboarding)/weights.tsx` shows a section only for equipment the profile has:

- **Dumbbells:** a two-way switch, Adjustable or Fixed set. Adjustable shows three `NumberField`s (min, max, step). Fixed shows chips for common weights (2.5, 5, 7.5 … 40 kg) to tick.
- **Kettlebells:** chips for common sizes (4, 6, 8, 12, 16, 20, 24, 28, 32 kg).
- **Loop bands:** chips for light, medium, heavy, extra heavy.
- Validate with `EquipmentProfile.shape.loads` through React Hook Form, as in M1 step 8, then PATCH `{ loads }` and Finish.

Add a refinement so adjustable dumbbells need `maxKg > minKg`, with the message "Max must be heavier than min".

You should see:

- [ ] A profile with kettlebells but no dumbbells shows only the kettlebell section
- [ ] Min 24, max 10 shows "Max must be heavier than min"
- [ ] After Finish, `onboardedAt` is set and the profile in Atlas has `loads`

6 · The exercise schema and the first 20 exercises

Add the schema to `@fit/shared`:

```
export const Pattern = z.enum(["squat", "hinge", "push", "pull", "carry",
"core"]);
export const Muscle = z.enum([
  "quads", "hamstrings", "glutes", "adductors", "calves", "chest",
"front_delts", "side_delts",
  "rear_delts", "lats", "upper_back", "traps", "biceps", "triceps",
"forearms", "abs", "obliques", "lower_back",
```

```
]);

export const Exercise = z.object({
  slug: z.string().regex(/^[a-z0-9-]+$/),            // stable ID, e.g.
"goblet-squat"
  name: z.string(),
  patterns: z.array(z.object({ pattern: Pattern, credit:
z.number().min(0).max(1) })).min(1),
  muscles: z.object({ primary: z.array(Muscle).min(1), secondary:
z.array(Muscle) }),
  equipmentOptions: z.array(z.array(EquipmentTag)).min(1), // any one option
works; [[]] = no equipment
  loadType: z.enum(["external", "bodyweight", "timed"]),
  unilateral: z.boolean(),
  secondsPerSet: z.number().int().positive(),          // including rest
  regression: z.string().nullable(),                  // slug of the easier
variant
  progression: z.string().nullable(),                 // slug of the harder
variant
  avoidWith: z.array(InjuryTag),
  cues: z.array(z.string()).max(3),
  videoUrl: z.url().nullable(),
});
export type Exercise = z.infer<typeof Exercise>;
```

`equipmentOptions` is a list of alternatives: a pull-up is `[["pull_up_bar"], ["rings"]]` (either works), a back squat is `[["barbell", "plates", "squat_rack"]]` (all three needed).

Put the data in `packages/engines/src/data/exercises.ts`, so planner tests in M5 can use it:

```
import type { Exercise } from "@fit/shared";

export const EXERCISES: Exercise[] = [
  {
    slug: "goblet-squat",
    name: "Goblet squat",
    patterns: [{ pattern: "squat", credit: 1 }],
    muscles: { primary: ["quads", "glutes"], secondary: ["adductors", "abs"]
},
    equipmentOptions: [["dumbbells"], ["kettlebells"]],
    loadType: "external",
    unilateral: false,
    secondsPerSet: 120,
    regression: "bodyweight-squat",
```

```
    progression: "back-squat",
    avoidWith: ["knee"],
    cues: ["Hold the weight at your chest", "Elbows inside knees", "Sit down
between your heels"],
    videoUrl: null,
  },
  // …
];
```

(`@fit/engines` needs `@fit/shared` as a dependency: `pnpm add "@fit/shared@workspace:*" --filter @fit/engines`.)

The first 20, chosen so every pattern has an option for every preset:

Pattern	Exercises
Squat	Bodyweight squat, goblet squat, back squat, Bulgarian split squat
Hinge	Single-leg glute bridge, kettlebell swing, dumbbell Romanian deadlift, barbell Romanian deadlift
Push	Incline push-up, push-up, dumbbell bench press, barbell bench press, dumbbell overhead press
Pull	Inverted row, pull-up, one-arm dumbbell row, band pull-apart
Core	Plank, dead bug
Carry	Farmer's carry

Then a seed script, `apps/api/scripts/seed-exercises.ts`:

```
import { Exercise } from "@fit/shared";
import { EXERCISES } from "@fit/engines";
import { client, db } from "../src/db";

const exercises = db.collection<Exercise>("exercises");
await exercises.createIndex({ slug: 1 }, { unique: true });

for (const raw of EXERCISES) {
  const exercise = Exercise.parse(raw); // refuse to seed anything invalid
  await exercises.replaceOne({ slug: exercise.slug }, exercise, { upsert:
true });
}

console.log(`Seeded ${EXERCISES.length} exercises`);
await client.close();
```

Add `"seed:exercises": "tsx --env-file=.env scripts/seed-exercises.ts"` to the API's scripts, `@fit/engines` as an API dependency, and `"@fit/engines"` to `noExternal` in `tsup.config.ts`. `replaceOne` with `upsert` makes the script safe to run any number of times: it updates existing exercises and adds new ones.

You should see:

- [ ] `pnpm --filter @fit/api seed:exercises` prints "Seeded 20 exercises"
- [ ] Running it again still shows 20 documents in Atlas, not 40
- [ ] A typo like `pattern: "sqaut"` fails `pnpm typecheck` before you can seed it

7 · `isAvailable()` with unit tests

`packages/engines/src/equipment.ts`:

```
import type { EquipmentProfile, EquipmentTag, Exercise, InjuryTag } from
"@fit/shared";

// Owning one item means owning another, e.g. a power rack includes a pull-up
bar
const IMPLIES: Partial<Record<EquipmentTag, EquipmentTag[]>> = {
  power_rack: ["squat_rack", "pull_up_bar"],
  adjustable_bench: ["flat_bench"],
};

export function expandTags(tags: EquipmentTag[]): Set<EquipmentTag> {
  const owned = new Set(tags);
  for (const tag of tags) IMPLIES[tag]?.forEach((extra) => owned.add(extra));
  return owned;
```

```
}

export function isAvailable(exercise: Pick<Exercise, "equipmentOptions">,
profile: Pick<EquipmentProfile, "tags">): boolean {
  const owned = expandTags(profile.tags);
  return exercise.equipmentOptions.some((option) => option.every((tag) =>
owned.has(tag)));
}

export function isSafeFor(exercise: Pick<Exercise, "avoidWith">, injuries:
InjuryTag[]): boolean {
  return !exercise.avoidWith.some((tag) => injuries.includes(tag));
}
```

Export both from `src/index.ts`, then write `equipment.test.ts` covering:

- a bodyweight exercise (`[[]]`) is available with no equipment
- a pull-up is available with rings but no bar
- a back squat is unavailable with a barbell but no rack
- a power rack counts as a squat rack (the `IMPLIES` rule)
- `isSafeFor` rejects a squat for someone with a knee injury

Add a **library integrity test** too, `data/exercises.test.ts`, which checks the data rather than code:

- every slug is unique, and every `regression` and `progression` points to a slug that exists
- every exercise passes `Exercise.parse`
- for every preset in `PRESETS`, every pattern except carry has at least one available exercise

That last test is what stops you shipping a library where a bodyweight-only user has no way to train pull.

You should see:

- [ ] `pnpm test` passes, with the new tests listed
- [ ] Deleting the inverted row from the data makes the preset test fail for "Bodyweight only" (then restore it)

8 · Exercise list screen

- API: `src/routes/exercises.ts`, a plugin with `GET /exercises` (all, sorted by name) and `GET /exercises/:slug` (404 if unknown), both behind `requireAuth`.

- App: `src/app/(app)/exercises/index.tsx`. Fetch `/me` and `/exercises`, filter with `isAvailable(exercise, primaryProfile)` (the same function, imported from `@fit/engines`), and show them in a `SectionList` grouped by each exercise's first pattern.
- A **Show all** switch turns the filter off, and unavailable exercises appear greyed out with "Needs: …" underneath.
- Add `@fit/engines` as a mobile dependency and a link to this screen from home.

You should see:

- [ ] With a "Dumbbells + bench" home profile, the list shows the goblet squat but not the back squat
- [ ] Show all reveals the back squat, greyed out, with "Needs: Barbell, Plates, Squat rack"
- [ ] Changing your profile's equipment (from the profile screen, or Atlas) changes the list after a refresh

9 · Exercise detail screen

`src/app/(app)/exercises/[slug].tsx`. The square brackets make it a **dynamic route**: `/exercises/goblet-squat` opens it with `slug = "goblet-squat"`, read with `useLocalSearchParams()`. Show:

- name, patterns and primary muscles
- equipment options in words ("Dumbbells, or kettlebells")
- the cues as a numbered list
- a grey 16:9 box saying "Video coming soon"
- **Easier** and **Harder** buttons linking to the regression and progression

You should see:

- [ ] Tapping an exercise in the list opens its detail screen
- [ ] Easier and Harder walk the ladder (bodyweight squat → goblet squat → back squat)
- [ ] An unknown slug in the URL shows "Exercise not found" rather than crashing

10 · Grow the library to about 80

Add exercises in batches of 10–15, re-running the tests and the seed script each time. Aim for:

- **Bodyweight ladders** for push (incline → push-up → decline → archer), pull (inverted row → negatives → pull-up) and squat (squat → split squat → shrimp or pistol), since these are how home users progress once weights run out.
- **Ring and band variants**, so those presets get real choice.

- **Single-leg and single-arm options**, useful when dumbbells are too light.
- **More core and carry** (side plank, hanging knee raise, suitcase carry).

Write cues in your own words, two or three short commands each, and keep `secondsPerSet` realistic (about 90 seconds for accessories, 150–180 for heavy compound lifts), because the planner uses it to fit sessions into a time cap.

You should see:

- [ ] The integrity test passes with every preset covering every pattern at least twice
- [ ] Every preset sees at least 15 exercises in the list
- [ ] `pnpm lint`, `pnpm typecheck` and `pnpm test` pass, and CI is green

M3 · Log a workout by hand

Done when: you can log a full gym session, with no signal, and see it in your history.

Design choices for M3: every set gets an ID made on the phone (`clientId`), so sending the same set twice never saves it twice; sets are saved on the phone first and synced in batches, so a gym with no signal loses nothing; a workout is simply all the sets that share a `sessionId` (M5 adds planned sessions on top). Two new app libraries arrive: **TanStack Query** for fetching (it works like RTK Query: cached queries, `isLoading`, invalidation) and **Zustand** for the in-progress workout (a tiny Redux-like store with no boilerplate).

1 · The set schema in `@fit/shared`

Create `packages/shared/src/workout.ts` and export it from `index.ts`:

```
import { z } from "zod";

export const SetLogInput = z
  .object({
    clientId: z.uuid(),                // made on the phone; makes retries
safe
    sessionId: z.uuid(),               // groups sets into one workout
    exerciseSlug: z.string(),
    performedAt: z.iso.datetime(),
    reps: z.number().int().min(0).max(100).nullable(),
    durationSec: z.number().int().positive().max(3600).nullable(), // for
timed exercises
    loadKg: z.number().min(0).max(500).nullable(),              // null =
bodyweight
    rpe: z.number().min(5).max(10).multipleOf(0.5).nullable(),
  })
  .refine((set) => set.reps !== null || set.durationSec !== null, "A set
```

```
needs reps or a time");
export type SetLogInput = z.infer<typeof SetLogInput>;

export const WorkoutSummary = z.object({
  sessionId: z.string(),
  startedAt: z.iso.datetime(),
  endedAt: z.iso.datetime(),
  setCount: z.number(),
  exerciseSlugs: z.array(z.string()),
});
export type WorkoutSummary = z.infer<typeof WorkoutSummary>;
```

RPE (rate of perceived exertion) is how hard a set felt from 1–10, where 10 means no more reps were possible; the app only offers 5–10 because easier sets don't need rating. It's optional now and becomes important for progression in M6.

You should see:

- [ ] `pnpm typecheck` passes
- [ ] A scratch `SetLogInput.safeParse` with both `reps` and `durationSec` null fails with "A set needs reps or a time"

2 · Save sets (API)

`apps/api/src/sets.ts`:

```
import { db } from "./db";

type SetLogDoc = {
  userId: string; // the Clerk user ID
  clientId: string;
  sessionId: string;
  exerciseSlug: string;
  performedAt: Date;
  reps: number | null;
  durationSec: number | null;
  loadKg: number | null;
  rpe: number | null;
  createdAt: Date;
};

export const setLogs = db.collection<SetLogDoc>("setLogs");

export async function ensureSetIndexes() {
  await setLogs.createIndex({ userId: 1, clientId: 1 }, { unique: true });
// no duplicates
```

```
  await setLogs.createIndex({ userId: 1, exerciseSlug: 1, performedAt: -1 });
// history per exercise
  await setLogs.createIndex({ userId: 1, sessionId: 1 });
// one workout
}
```

Call `ensureSetIndexes()` at startup, and add `src/routes/sets.ts`, registered like the `/me` routes:

```
export const setRoutes: FastifyPluginAsyncZod = async (app) => {
  app.addHook("preHandler", requireAuth);

  app.post("/sets", { schema: { body: z.array(SetLogInput).min(1).max(200) }
}, async (req, reply) => {
    const userId = getAuth(req).userId!;

    const slugs = [...new Set(req.body.map((s) => s.exerciseSlug))];
    const known = await exercises.countDocuments({ slug: { $in: slugs } });
    if (known !== slugs.length) return reply.code(400).send({ error: "unknown
exercise" });

    const result = await setLogs.bulkWrite(
      req.body.map((set) => ({
        updateOne: {
          filter: { userId, clientId: set.clientId },
          update: {
            $setOnInsert: { ...set, userId, performedAt: new
Date(set.performedAt), createdAt: new Date() },
          },
          upsert: true,
        },
      })),
      { ordered: false }
    );
    return { received: req.body.length, saved: result.upsertedCount };
  });
};
```

The route takes a **list** of sets, because the phone may have several queued up after time offline. `bulkWrite` sends them to MongoDB in one round trip. Each set is an upsert keyed on `clientId` with `$setOnInsert`: a new set is inserted, and a set the server already has is left alone. That's what makes retrying safe. (Export the `exercises` collection from a small `src/exercises.ts`, the same way as `users`.)

You should see:

- [ ] Sending the same set twice returns `saved: 1` then `saved: 0`, and Atlas has one document
- [ ] A set for exercise `"made-up"` returns 400 "unknown exercise"
- [ ] The three indexes appear in Atlas's Indexes tab for `setLogs`

3 · Read workouts back (API)

Add to `routes/sets.ts`:

```
app.get("/workouts", async (req) => {
  const rows = await setLogs
    .aggregate<{ _id: string; startedAt: Date; endedAt: Date; setCount:
number; exerciseSlugs: string[] }>([
      { $match: { userId: getAuth(req).userId! } },
      {
        $group: {
          _id: "$sessionId",
          startedAt: { $min: "$performedAt" },
          endedAt: { $max: "$performedAt" },
          setCount: { $sum: 1 },
          exerciseSlugs: { $addToSet: "$exerciseSlug" },
        },
      },
      { $sort: { startedAt: -1 } },
      { $limit: 30 },
    ])
    .toArray();

  return rows.map((row) =>
    WorkoutSummary.parse({ ...row, sessionId: row._id, startedAt:
row.startedAt.toISOString(), endedAt: row.endedAt.toISOString() })
  );
});

app.get("/workouts/:sessionId", { schema: { params: z.object({ sessionId:
z.string() }) } }, async (req) => {
  return setLogs
    .find({ userId: getAuth(req).userId!, sessionId: req.params.sessionId },
{ projection: { _id: 0, userId: 0 } })
    .sort({ performedAt: 1 })
    .toArray();
});
```

The **aggregation pipeline** works like a series of SQL steps: `$match` filters to your sets, `$group` collapses each `sessionId` into one row with its first and last set times, a count, and the list of exercises, then `$sort` and `$limit` keep the 30 most recent.

You should see:

- [ ] After logging sets for two different `sessionId`s, `/workouts` returns two summaries, newest first
- [ ] `/workouts/:sessionId` returns that workout's sets in the order they were done

4 · Add TanStack Query to the app

```
pnpm --filter @fit/mobile add @tanstack/react-query
```

In `src/app/_layout.tsx`, wrap the app inside `ClerkProvider`:

```
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";

const queryClient = new QueryClient();

// inside RootLayout:
<ClerkProvider publishableKey={publishableKey} tokenCache={tokenCache}>
  <QueryClientProvider client={queryClient}>
    <Slot />
  </QueryClientProvider>
</ClerkProvider>
```

Then fetching becomes one hook, with caching and loading state built in, much like RTK Query's generated hooks:

```
const exercisesQuery = useQuery({
  queryKey: ["exercises"],
  queryFn: async () => z.array(Exercise).parse(await api("/exercises")),
  staleTime: Infinity, // the library rarely changes; fetch once per app
launch
});
```

Also wrap the function returned by `useApi` in `useCallback(…, [getToken])`, so it stays the same between renders; step 8 relies on that.

You should see:

- [ ] Switch the M2 exercise list to `useQuery`: it loads as before, and returning to it later is instant (cached)

5 · The workout store (Zustand)

```
pnpm --filter @fit/mobile add zustand
pnpm --filter @fit/mobile exec expo install @react-native-async-
storage/async-storage
```

`src/lib/workoutStore.ts` holds the active workout and every set not yet confirmed by the server. The `persist` middleware saves it to the phone's storage after every change, so closing the app mid-workout loses nothing:

```
import AsyncStorage from "@react-native-async-storage/async-storage";
import type { SetLogInput } from "@fit/shared";
import * as Crypto from "expo-crypto";
import { create } from "zustand";
import { createJSONStorage, persist } from "zustand/middleware";

type ActiveWorkout = { sessionId: string; startedAt: string; exerciseSlugs:
string[] };
export type LoggedSet = SetLogInput & { synced: boolean };
type NewSet = Omit<SetLogInput, "clientId" | "sessionId" | "performedAt">;

type WorkoutState = {
  active: ActiveWorkout | null;
  sets: LoggedSet[];
  start: () => void;
  addExercise: (slug: string) => void;
  logSet: (set: NewSet) => void;
  markSynced: (clientIds: string[]) => void;
  finish: () => void;
};

export const useWorkoutStore = create<WorkoutState>()(
  persist(
    (set, get) => ({
      active: null,
      sets: [],

      start: () =>
        set({ active: { sessionId: Crypto.randomUUID(), startedAt: new
Date().toISOString(), exerciseSlugs: [] } }),

      addExercise: (slug) =>
        set((s) =>
          s.active && !s.active.exerciseSlugs.includes(slug)
            ? { active: { ...s.active, exerciseSlugs:
[...s.active.exerciseSlugs, slug] } }
```

```
            : s
        ),

      logSet: (newSet) => {
        const active = get().active;
        if (!active) return;
        const logged: LoggedSet = {
          ...newSet,
          clientId: Crypto.randomUUID(),
          sessionId: active.sessionId,
          performedAt: new Date().toISOString(),
          synced: false,
        };
        set((s) => ({ sets: [...s.sets, logged] }));
      },

      markSynced: (ids) =>
        set((s) => ({ sets: s.sets.map((x) => (ids.includes(x.clientId) ? {
...x, synced: true } : x)) })),

      // Keep anything not yet synced; drop what the server already has
      finish: () => set((s) => ({ active: null, sets: s.sets.filter((x) =>
!x.synced) })),
    }),
    { name: "workout", storage: createJSONStorage(() => AsyncStorage) }
  )
);
```

In a component, read only what you need, e.g. `const active = useWorkoutStore((s) => s.active)`; it re-renders only when that piece changes, like a Redux selector. Outside React (step 8), use `useWorkoutStore.getState()`.

You should see:

- [ ] Start a workout, log a set, fully close Expo Go and reopen: the workout and the set are still there

6 · The workout screen

Files:

- `src/app/(app)/workout/index.tsx`: the live workout
- `src/app/(app)/workout/add-exercise.tsx`: the M2 list again, where tapping an exercise calls `addExercise(slug)` then `router.back()`
- `src/components/SetEntry.tsx`: the logging row for one exercise

The workout screen:

- **No active workout:** a big **Start workout** button calling `start()`.
- **Active:** elapsed time at the top, then one section per exercise in `active.exerciseSlugs`, each listing its logged sets ("3 × 8 @ 20 kg, RPE 8") and a `SetEntry` row; then **Add exercise** and **Finish workout** at the bottom.
- Call `useKeepAwake()` from `expo-keep-awake` so the phone doesn't lock between sets.

`SetEntry` is built for one tap:

```
type Props = { exercise: Exercise; previous?: LoggedSet; onLog: (set: NewSet)
=> void };

export function SetEntry({ exercise, previous, onLog }: Props) {
  const timed = exercise.loadType === "timed";
  const [reps, setReps] = useState(previous?.reps ?? 8);
  const [durationSec, setDurationSec] = useState(previous?.durationSec ??
30);
  const [loadKg, setLoadKg] = useState<number | undefined>(previous?.loadKg
?? undefined);
  const [rpe, setRpe] = useState<number | null>(null);

  const log = () =>
    onLog({
      exerciseSlug: exercise.slug,
      reps: timed ? null : reps,
      durationSec: timed ? durationSec : null,
      loadKg: exercise.loadType === "external" ? (loadKg ?? null) : null,
      rpe,
    });

  // Render: -/+ steppers for reps (or seconds), a NumberField for kg on
loaded
  // exercises, a row of RPE chips (6, 7, 8, 9, 10), and a large "Log set"
button.
}
```

- **`previous` is the last set of that exercise in this workout**, so set 2 starts filled with set 1's numbers and logging a repeat is a single tap. (M5 replaces this with planned targets.)
- **Steppers, not typing, for reps:** big − and + buttons are easier with sweaty hands.
- **Log set** calls `logSet`, then `flushQueue` (step 8), then starts the rest timer (step 7).

- **Finish workout** calls `flushQueue`, then `finish()`, then goes to the workout's history page.

You should see:

- [ ] You can start a workout, add three exercises and log sets for each
- [ ] Set 2 is pre-filled with set 1's numbers
- [ ] Timed exercises (plank) ask for seconds; bodyweight ones don't ask for kg
- [ ] The screen stays on during the workout

7 · Rest timer

`src/components/RestTimer.tsx` counts down to a fixed end time rather than subtracting a second at a time, so it stays correct even if the app is in the background for a while:

```
import { useEffect, useRef, useState } from "react";
import { Button, Text, Vibration, View } from "react-native";

type Props = { endsAt: number; onAdjust: (deltaSec: number) => void; onDone:
() => void };

export function RestTimer({ endsAt, onAdjust, onDone }: Props) {
  const [now, setNow] = useState(Date.now());
  const fired = useRef(false);

  useEffect(() => {
    const id = setInterval(() => setNow(Date.now()), 250);
    return () => clearInterval(id);
  }, []);

  const remaining = Math.max(0, Math.ceil((endsAt - now) / 1000));

  useEffect(() => {
    if (remaining === 0 && !fired.current) {
      fired.current = true;
      Vibration.vibrate(400);
      onDone();
    }
  }, [remaining, onDone]);

  const mm = Math.floor(remaining / 60);
  const ss = String(remaining % 60).padStart(2, "0");

  return (
    <View style={{ alignItems: "center", gap: 8 }}>
      <Text style={{ fontSize: 48, fontVariant: ["tabular-nums"] }}>{mm}:{ss}
```

```
</Text>
      <View style={{ flexDirection: "row", gap: 12 }}>
        <Button title="−15s" onPress={() => onAdjust(-15)} />
        <Button title="+15s" onPress={() => onAdjust(15)} />
        <Button title="Skip" onPress={onDone} />
      </View>
    </View>
  );
}
```

The workout screen keeps `restEndsAt` in state and shows the timer while it's set. The default rest comes from the exercise data: `Math.max(45, exercise.secondsPerSet - 30)` seconds, so heavy lifts get about 2–2.5 minutes and accessories about a minute. Give `RestTimer` a `key={restEndsAt}` so a new countdown starts fresh.

The phone only vibrates if the app is open; an alert while the app is in the background needs local notifications, which arrive in M8.

You should see:

- [ ] Logging a goblet squat set starts a 1:30 countdown; a back squat starts 2:30
- [ ] +15s and −15s adjust it; Skip hides it
- [ ] Locking the phone for 30 seconds and returning shows the right time remaining
- [ ] The phone vibrates at 0:00

8 · Offline queue and sync

```
pnpm --filter @fit/mobile exec expo install @react-native-community/netinfo
```

`src/lib/sync.ts` sends every unsynced set in one request:

```
import { SetLogInput } from "@fit/shared";
import { useWorkoutStore } from "./workoutStore";

type Api = (path: string, init?: RequestInit) => Promise<unknown>;
let flushing = false;

export async function flushQueue(api: Api) {
  if (flushing) return; // one flush at a time
  const pending = useWorkoutStore.getState().sets.filter((s) => !s.synced);
  if (pending.length === 0) return;

  flushing = true;
  try {
    const body = pending.map((s) => SetLogInput.parse(s)); // parse strips
```

```
the local `synced` flag
    await api("/sets", { method: "POST", body: JSON.stringify(body) });
    useWorkoutStore.getState().markSynced(pending.map((s) => s.clientId));
  } catch {
    // Offline or server error: the sets stay queued and the next flush
retries them
  } finally {
    flushing = false;
  }
}
```

Then flush at the right moments, from a hook used in `(app)/_layout.tsx`:

```
import NetInfo from "@react-native-community/netinfo";
import { useEffect } from "react";
import { AppState } from "react-native";

export function useBackgroundSync() {
  const api = useApi();

  useEffect(() => {
    flushQueue(api); // on launch
    const stopNet = NetInfo.addEventListener((state) => {
      if (state.isConnected) flushQueue(api); // when signal returns
    });
    const appState = AppState.addEventListener("change", (status) => {
      if (status === "active") flushQueue(api); // when the app comes back to
the front
    });
    return () => {
      stopNet();
      appState.remove();
    };
  }, [api]);
}
```

Because the server ignores sets it already has (step 2), it doesn't matter if a flush is interrupted halfway and repeated.

You should see:

- [ ] With airplane mode on, logging sets works normally and each shows a small "not synced" dot
- [ ] Turning airplane mode off syncs them within a few seconds, and the dots disappear
- [ ] Atlas has exactly one document per set, even after toggling airplane mode several times mid-sync

- [ ] Stopping the API, logging sets, then restarting the API: they sync next time the app comes to the front

9 · Workout history

- `src/app/(app)/history/index.tsx`: `useQuery({ queryKey: ["workouts"], … })` on `/workouts`, shown as a list: date, duration (`endedAt − startedAt`), number of sets, and exercise names (look up slugs in the cached `["exercises"]` query).
- `src/app/(app)/history/[sessionId].tsx`: the sets from `/workouts/:sessionId` grouped by exercise, e.g. "Goblet squat: 8 × 20 kg, 8 × 20 kg, 7 × 20 kg".
- **Show unsynced workouts too:** merge any sets still in the store whose `sessionId` isn't in the server list, labelled "Waiting to sync", so history never looks like it lost a workout.
- After **Finish workout** succeeds, call `queryClient.invalidateQueries({ queryKey: ["workouts"] })` so history refetches, just like invalidating tags in RTK Query.
- **Pull to refresh** on the list: `FlatList`'s `refreshing` and `onRefresh` props, wired to `refetch()`.

You should see:

- [ ] A finished workout appears at the top of history with the right set count and duration
- [ ] Tapping it shows every set, grouped by exercise, in order
- [ ] A workout finished in airplane mode shows "Waiting to sync", then becomes normal once synced
- [ ] `pnpm lint`, `pnpm typecheck` and `pnpm test` pass, and CI is green

M4 · Baseline test

Done when: you've done both test sessions and every exercise in your library has a starting load or rep target.

- [ ] `chooseTests(profile)` picks one test per pattern from your equipment (unit tests)
- [ ] Epley e1RM function (unit tests)
- [ ] Test session A (lower body + core): guided work-up sets with a stop at RPE 9
- [ ] Test session B (upper body + optional conditioning)
- [ ] Save the `Assessment` document
- [ ] Ratio table, and seed `exerciseState` for every available exercise from it (unit tests)
- [ ] Results screen showing each test and your starting loads

M5 · Programs and the planner

Done when: the home screen shows tomorrow's session, built from your weekly budget, and you can switch its mode. Start training with the app from here.

Design choices for M5: the planner is a set of small pure functions in `@fit/engines`, each with its own tests, joined by one `planSession()` function; the API only loads data, calls it and saves the result. The week ledger is recalculated from your set logs rather than incremented, so a retried or late sync can never double-count. A planned session's `id` becomes the workout's `sessionId`, so logged sets link back to the plan automatically. This milestone assumes M4 has created `exerciseState` entries.

1 · Schemas in `@fit/shared`

Create `packages/shared/src/program.ts`:

```
export const BudgetLine = z.object({
  pattern: Pattern,
  setsPerWeek: z.number().int().min(0),
  repRange: z.tuple([z.number().int(), z.number().int()]),
  rpe: z.number().min(5).max(10),
});

export const Program = z.object({
  slug: z.string(),
  name: z.string(),
  blockWeeks: z.number().int().min(1),
  deloadWeek: z.number().int().min(1), // which week of the block is the
deload
  budget: z.array(BudgetLine),
  conditioningMinPerWeek: z.number().int().min(0),
});

export const PatternCounts = z.record(Pattern, z.number());

export const WeekLedger = z.object({
  weekStart: z.iso.date(),          // a Monday, in the user's time zone
  weekIndex: z.number().int(),      // 1-based week of the block
  isDeload: z.boolean(),
  targets: PatternCounts,
  done: PatternCounts,
  sessionsGoal: z.number().int(),
  sessionsDone: z.number().int(),
});

export const Mode = z.enum(["gym", "home", "bodyweight", "short",
```

```
"recovery"]);

export const SetTarget = z.object({
  targetReps: z.number().int().nullable(),
  targetDurationSec: z.number().int().nullable(),
  targetLoadKg: z.number().nullable(),
  targetRpe: z.number(),
  calibration: z.boolean(), // the feeler set for a new exercise
});

export const PlannedExercise = z.object({
  exerciseSlug: z.string(),
  pattern: Pattern,
  supersetGroup: z.string().nullable(), // "A", "B" … or null
  restSec: z.number().int(),
  sets: z.array(SetTarget),
});

export const PlannedSession = z.object({
  id: z.uuid(),                 // also the sessionId of the logged sets
  date: z.iso.date(),
  mode: Mode,
  status: z.enum(["planned", "in_progress", "done", "skipped"]),
  timeCapMin: z.number().int(),
  patternTargets: PatternCounts, // kept so a mode switch keeps the same
goals
  exercises: z.array(PlannedExercise),
  reasons: z.array(z.string()),
  estimatedMin: z.number(),
});

export const CheckIn = z.object({
  date: z.iso.date(),
  sleep: z.number().int().min(1).max(5),
  energy: z.number().int().min(1).max(5),
  soreness: z.number().int().min(1).max(5), // 5 = very sore
  note: z.string().max(280).optional(),
});
```

Also add `timeZone: z.string()` to `UserProfile`, filled from the phone at onboarding with `getCalendars()[0].timeZone` from `expo-localization`. Week boundaries and "today" depend on it: 11 pm in Vancouver is already tomorrow in UTC.

You should see:

- [ ] `pnpm typecheck` passes

- [ ] A `PlannedSession` with `mode: "swimming"` fails to parse

2 · The first program, seeded

In `packages/engines/src/data/programs.ts`:

```
export const PROGRAMS: Program[] = [
  {
    slug: "athletic-base",
    name: "Athletic base",
    blockWeeks: 5,
    deloadWeek: 5,
    budget: [
      { pattern: "squat", setsPerWeek: 9, repRange: [6, 10], rpe: 8 },
      { pattern: "hinge", setsPerWeek: 6, repRange: [6, 10], rpe: 8 },
      { pattern: "push", setsPerWeek: 12, repRange: [6, 12], rpe: 8 },
      { pattern: "pull", setsPerWeek: 12, repRange: [6, 12], rpe: 8 },
      { pattern: "core", setsPerWeek: 6, repRange: [8, 15], rpe: 7 },
      { pattern: "carry", setsPerWeek: 3, repRange: [1, 1], rpe: 7 },
    ],
    conditioningMinPerWeek: 90,
  },
];
```

The budget is the same whatever your days per week; more days just spreads it thinner. Extend the M2 seed script to upsert programs by slug as well.

You should see:

- [ ] The seed script prints the program count, and Atlas shows one document in `programs`

3 · Start a block (API)

Add `src/blocks.ts` with `blocks` and `weeks` collections, and an engine helper for dates:

```
// packages/engines/src/dates.ts — uses date-fns and @date-fns/tz
export function localDate(now: Date, timeZone: string): string     // "2026-
11-02"
export function weekStartOf(date: string): string                  // the
Monday on or before
export function weekIndexFor(blockStart: string, date: string): number // 1,
2, 3 …
```

Then `POST /blocks/start`, called by the app when M4's baseline results screen is dismissed:

- Ends any active block (`status: "ended"`), then inserts `{ userId, programSlug: "athletic-base", startDate: <this week's Monday>, status: "active" }`.
- Returns the new block. Starting on the Monday of the current week keeps week numbering simple.

And `GET /weeks/current`, which finds the active block, works out this week's index, and **creates the ledger if it doesn't exist yet**: targets from the program budget (halved, rounded up, on the deload week), `sessionsGoal` from the user's `daysPerWeek`, and `done` from step 4.

You should see:

- [ ] Starting a block creates one active block in Atlas; starting another ends the first
- [ ] `GET /weeks/current` creates this week's ledger on first call and returns the same one after
- [ ] Unit tests for `weekStartOf`: a Sunday returns the Monday six days earlier; a Monday returns itself

4 · Count the week from the logs

A pure function in `packages/engines/src/ledger.ts`:

```
export function tallyWeek(
  sets: { exerciseSlug: string; sessionId: string }[],
  exercises: Map<string, Exercise>
): { done: Record<Pattern, number>; sessionsDone: number } {
  const done = Object.fromEntries(Pattern.options.map((p) => [p, 0])) as
Record<Pattern, number>;
  for (const set of sets) {
    for (const { pattern, credit } of
exercises.get(set.exerciseSlug)?.patterns ?? []) {
      done[pattern] += credit;
    }
  }
  return { done, sessionsDone: new Set(sets.map((s) => s.sessionId)).size };
}
```

In the API, `recomputeWeek(userId, weekStart)` loads that week's sets, runs `tallyWeek`, and `$set`s the ledger's `done` and `sessionsDone`. Call it at the end of `POST /sets`. Because it recounts from scratch, a retried sync or a set arriving days late always produces the right totals.

You should see unit tests proving:

- [ ] Three goblet squat sets count 3 squat

- [ ] A renegade row counts 1 pull and 0.5 core
- [ ] Sets from two different `sessionId`s give `sessionsDone: 2`
- [ ] An unknown slug is ignored rather than crashing

And in the app: logging sets and checking `GET /weeks/current` shows `done` going up.

5 · Planner: tomorrow's share of the budget

From here, each planner step is a file in `packages/engines/src/planner/` with a test file beside it.

```
// share.ts
export function sessionShare(
  targets: Record<Pattern, number>,
  done: Record<Pattern, number>,
  sessionsLeft: number,
  maxSetsPerPattern = 6
): Record<Pattern, number> {
  const left = Math.max(1, sessionsLeft);
  return mapPatterns((p) => {
    const remaining = Math.max(0, targets[p] - done[p]);
    return Math.min(maxSetsPerPattern, Math.ceil(remaining / left));
  });
}
```

`mapPatterns` is a tiny helper that builds a record with a value for every pattern. Dividing the remainder by the sessions left means a missed day raises the next sessions a little rather than piling the whole backlog into one; the cap stops any one pattern taking over a session.

Tests:

- [ ] 12 push sets, 0 done, 3 sessions left → 4
- [ ] 12 push, 4 done, 2 sessions left → 4 (not 8)
- [ ] 12 push, 0 done, 1 session left → 6 (the cap)
- [ ] Done more than target → 0, never negative

6 · Planner: recovery filter

```
// fatigue.ts
export function patternFatigue(
  recent: { exerciseSlug: string; performedAt: Date }[],
  activityFatigue: Partial<Record<Pattern, number>>, // from M10; {} for now
  exercises: Map<string, Exercise>,
  now: Date
```

```
): Record<Pattern, number> {
  const fatigue = mapPatterns((p) => activityFatigue[p] ?? 0);
  for (const set of recent) {
    const hoursAgo = (now.getTime() - set.performedAt.getTime()) / 3_600_000;
    if (hoursAgo >= 48) continue;
    const weight = 1 - hoursAgo / 48; // a set counts fully now, nothing
after 48h
    for (const { pattern, credit } of
exercises.get(set.exerciseSlug)?.patterns ?? []) {
      fatigue[pattern] += credit * weight;
    }
  }
  return fatigue;
}

export const FATIGUE_LIMIT = 4; // about 4 hard sets in the last day
```

The planner sets a pattern's share to 0 when its fatigue is over the limit and adds the reason code `"<pattern>_recovering"`. A linear 48-hour decay is a simple first model; you can tune the limit once you've trained with it.

Tests:

- [ ] 6 squat sets done 12 hours ago → squat fatigue 4.5, over the limit
- [ ] The same 6 sets 40 hours ago → 1.0, fine
- [ ] Sets from 3 days ago → 0

7 · Daily check-in and readiness

API: a `checkIns` collection with a unique index on `{ userId, date }`, and `PUT /check-ins/:date` (an upsert, so editing today's check-in replaces it).

Engine:

```
// readiness.ts
export function readiness(checkIn: CheckIn | null): { setFactor: number;
rpeDrop: number; reason?: string } {
  if (!checkIn) return { setFactor: 1, rpeDrop: 0 };
  const score = checkIn.sleep + checkIn.energy + (6 - checkIn.soreness); // 3
(awful) to 15 (great)
  if (score <= 6) return { setFactor: 0.7, rpeDrop: 1, reason:
"low_readiness" };
  if (score <= 9) return { setFactor: 0.85, rpeDrop: 0.5, reason:
"moderate_readiness" };
  return { setFactor: 1, rpeDrop: 0 };
}
```

The planner multiplies each pattern's share by `setFactor` (rounding, minimum 1 for patterns it keeps) and lowers every target RPE by `rpeDrop`.

Tests:

- [ ] No check-in → unchanged
- [ ] Sleep 1, energy 2, soreness 5 → factor 0.7, RPE drop 1, reason `low_readiness`
- [ ] All 4s → factor 1

8 · Planner: choose exercises

```
// select.ts
export function chooseExercises(
  share: Record<Pattern, number>,
  candidates: Exercise[],            // already filtered by isAvailable and
isSafeFor
  states: Map<string, ExerciseState>,
  recentSlugs: string[]              // exercises done in the last two weeks,
newest first
): { exercise: Exercise; pattern: Pattern; sets: number }[]
```

Rules, applied per pattern with a share above 0, biggest share first:

1. Candidates are exercises whose **main** pattern (credit 1) matches.
2. Score each: **+3** if it has an `exerciseState` (the planner knows what weight to give), **+2** if done in the last two weeks (consistency makes progress measurable), **+1** if it's a compound lift with a high `secondsPerSet`. Ties break by slug, so the same inputs always give the same plan.
3. The top exercise gets up to 4 sets; anything over 4 goes to the second-best exercise, for variety without chaos.

Tests:

- [ ] Given the goblet squat with state and the back squat without, the goblet squat wins
- [ ] A push share of 6 gives two push exercises, 4 + 2 sets
- [ ] Same inputs twice → identical output
- [ ] A pattern with no candidates is skipped with the reason code `"no_<pattern>_exercise"`

9 · Planner: fit to the time cap

```
// fit.ts
export function fitToTime(
  picks: Pick[],
  timeCapMin: number
): { picks: Pick[]; supersets: Map<string, string>; estimatedMin: number;
reasons: string[] }
```

- **Estimate:** 5 minutes of warm-up plus each exercise's `sets × secondsPerSet`.
- **Over the cap? First, supersets:** pair exercises that don't compete (push with pull, squat with core, hinge with push). A pair takes about `sets × (longer secondsPerSet + 30)` seconds instead of the two added together.
- **Still over? Trim:** remove one set at a time from the pattern with the smallest share, never going below the top two patterns' first exercise, and add `"trimmed_for_time"`.

Tests:

- [ ] A session estimated at 40 minutes with a 50-minute cap is unchanged
- [ ] At a 30-minute cap, push and pull become superset A
- [ ] At a 15-minute cap, only the two biggest patterns remain
- [ ] The estimate never exceeds the cap unless only the two protected exercises are left

10 · Planner: prescribe sets, and `planSession()`

```
// prescribe.ts
export function prescribe(
  exercise: Exercise,
  state: ExerciseState | undefined,
  sets: number,
  budget: BudgetLine,
  rpeDrop: number
): SetTarget[]
```

- **With state:** every set targets `state.loadKg` (null for bodyweight) and the reps `state` says to aim for next, at `state.targetRpe - rpeDrop`.
- **Without state:** set 1 is a **calibration** set (no load given, top of the rep range, RPE 6), and the rest target RPE 8. After set 1, the app asks how it felt and fills in the rest (step 13).
- **Timed exercises** use `targetDurationSec` instead of reps.

Then the function that joins everything:

```
// planner/index.ts
export function planSession(input: PlannerInput): Omit<PlannedSession, "id" |
"status"> {
  // 1. share       = sessionShare(week.targets, week.done, sessionsLeft)
  // 2. recovery    = zero out patterns over FATIGUE_LIMIT, add reasons
  // 3. readiness   = scale the share, note rpeDrop
  // 4. candidates  = exercises filtered by isAvailable(mode's profile) and
isSafeFor(injuries)
  // 5. picks       = chooseExercises(...)
  // 6. fitted      = fitToTime(picks, timeCapFor(mode))
  // 7. sets        = prescribe(...) for each pick
  // 8. reasons     = collected from every step, e.g. "pull_behind_budget"
when pull has the biggest share
}
```

`PlannerInput` holds everything the planner needs, so the function never touches the database. Mode decides two things: which equipment profile filters exercises (`gym` and `home` use those profiles; `bodyweight` and `recovery` use no equipment) and the time cap (`short` = 20 minutes; `recovery` plans a 30-minute walk or mobility session instead of strength work).

Tests, using the real exercise library and program:

- [ ] A fresh week, home profile with dumbbells, 50-minute cap: every pattern with a share appears, and the estimate is under 50 minutes
- [ ] Switching mode to bodyweight keeps `patternTargets` identical but every exercise is equipment-free
- [ ] After a heavy leg day yesterday, tomorrow's plan has no squat or hinge, with `squat_recovering` in reasons
- [ ] A full week already done gives a short optional session with the reason `budget_met`

11 · Serve sessions (API)

- **`GET /sessions/:date`**: returns that day's planned session, **planning it first if none exists** (gather the inputs, call `planSession`, give it a new UUID, save it). The first look at today's card is what creates the plan; M8's evening job will pre-plan tomorrow the same way.
- **`POST /sessions/:id/mode`** with `{ mode }`: re-runs steps 4–7 of the planner using the session's stored `patternTargets`, and replaces `exercises`. Refuse with 409 if the session is already `done`. Sets already logged keep counting in the week ledger, because they belong to the same `sessionId`.
- **`POST /sessions/:id/status`** with `{ status }`: for starting, finishing and skipping.

- A unique index on `{ userId, date }` in `plannedSessions` stops two plans for one day.

You should see:

- [ ] Calling `GET /sessions/<today>` twice returns the same session `id`
- [ ] Switching mode returns a different exercise list with the same `patternTargets`
- [ ] Switching mode on a finished session returns 409

12 · Today card on the home screen

`src/components/TodayCard.tsx`, fetching `GET /sessions/<today>` with `useQuery` (use the profile's time zone to work out today's date):

- Headline from the patterns, e.g. "Upper body · 45 min", and the one-line reason ("Legs are recovering from Monday").
- The exercise list in brief: "Push-up 4 × 10 · One-arm row 4 × 10 …".
- A **Start** button, and the mode chip (step 14).
- Underneath: this week's progress from `GET /weeks/current` ("Session 2 of 3 · Push 8/12") and, from M8 on, tomorrow's preview.

You should see:

- [ ] Opening the app shows today's planned session without pressing anything
- [ ] The weekly progress line updates after finishing a workout

13 · Start a planned session

Extend the M3 workout store with `startPlanned(session: PlannedSession)`: it uses `session.id` as the `sessionId` and keeps the targets alongside the exercises. In the workout screen:

- Each `SetEntry` is **pre-filled from the target** instead of the previous set, so a set done as planned is one tap on **Log set**; nudge ± for anything different.
- Rest uses the planned `restSec`. Supersets show both exercises together and alternate between them, with the rest after the pair.
- **Calibration sets:** after logging set 1 of a new exercise, ask "How hard was that?" (RPE). Work out an estimate with `e1rm(load, reps + (10 - rpe))` from `@fit/engines`, then fill the remaining sets at about 75% of it, rounded to weights the profile has.
- Starting calls `POST /sessions/:id/status` with `in_progress`; **Finish** marks it `done` and invalidates the `["session"]` and `["week"]` queries.
- Logging an extra exercise that wasn't planned still works and still counts.

You should see:

- [ ] Doing a whole planned session needs one tap per set when you match the targets
- [ ] Atlas shows the logged sets with the planned session's `id` as `sessionId`
- [ ] A calibration set at 12 kg × 12, RPE 6, fills the rest at a sensible heavier weight

14 · The mode chip

A row of chips on the Today card and at the top of the workout screen: **Gym · Home · Bodyweight · Short · Recovery** (show Gym and Home only if you have those profiles). Tapping one calls `POST /sessions/:id/mode`, shows a spinner on the card, and replaces the session in the query cache with the response (`queryClient.setQueryData`).

You should see:

- [ ] Switching to Bodyweight replaces dumbbell exercises with bodyweight ones for the same patterns
- [ ] Short brings the estimate to 20 minutes or less
- [ ] Switching mid-workout keeps the sets you already logged

15 · Daily check-in screen

`src/app/(app)/check-in.tsx`: three rows of five large faces or numbers (sleep, energy, soreness) and an optional note, saved with `PUT /check-ins/<today>`. Show it as a small card at the top of home until done for the day, not as a blocking pop-up. Saving invalidates today's session query, so if today's plan hasn't been started, the API re-plans it with the new readiness (have `PUT /check-ins` delete an unstarted plan for that date).

You should see:

- [ ] A low check-in (sleep 1, energy 2, soreness 5) turns today's plan into fewer sets with lower RPE targets, and the reason says so
- [ ] A check-in after starting the workout doesn't change it
- [ ] The card disappears once you've checked in today
- [ ] `pnpm lint`, `pnpm typecheck` and `pnpm test` pass, CI is green, and you can train a full week from the app

M6 · Progression

Done when: after a few weeks, loads go up on their own and a deload week arrives on schedule.

- [ ] Double progression after each session (unit tests)
- [ ] Bodyweight ladder progression (unit tests)
- [ ] Load ceilings for home equipment, with fallback progressions (unit tests)

- [ ] Calibration set for exercises with no history
- [ ] Miss twice, drop the load (unit tests)
- [ ] Deload week handling
- [ ] End-of-block retest prompt, reusing the M4 test screens
- [ ] PR detection from e1RM and max reps

M7 · Nutrition logging

Done when: you can log a meal by photo or a few words in under 15 seconds and see kcal, protein and fibre for the day.

- [ ] Targets: Mifflin-St Jeor kcal, protein and fibre minimum (unit tests)
- [ ] `nutritionTargets` document written at the end of onboarding
- [ ] Home screen bars for kcal, protein and fibre
- [ ] USDA FoodData Central client with a local cache
- [ ] Manual food search and log by grams (the fallback for everything else)
- [ ] LLM text parsing to a strict JSON schema, validated with zod
- [ ] Match parsed items to the food database; LLM numbers only as a low-confidence fallback
- [ ] Text meal logging with a confirm screen and portion slider
- [ ] Private photo bucket (S3 or R2) with signed upload URLs
- [ ] Photo meal logging through the same pipeline
- [ ] Saved foods: save from the confirm screen, one-tap relog
- [ ] `dailySummaries` updated on every meal

M8 · The daily loop

Done when: each evening your phone tells you tomorrow's session and what to fix in your eating.

- [ ] In-process event bus for SetLogged, SessionCompleted, MealLogged, CheckIn
- [ ] Move the ledger and summary updates onto events
- [ ] Scheduler (Agenda) with an evening job per user at about 8 pm local time
- [ ] LLM summary from reason codes and the day's nutrition gap
- [ ] Switch to an EAS development build (needed from here for notifications and health data)
- [ ] Push notifications with the summary (expo-notifications)
- [ ] Sunday job: recalculate targets and write a weekly review

- [ ] Weekly review screen

M9 · Rewards

Done when: finishing a session, hitting protein or a PR gives you XP and a moment of celebration.

- [ ] `xpLedger` and `userProgress`; award XP from events (unit tests)
- [ ] XP bar and level on the home screen
- [ ] Weekly goal and weekly streak (unit tests)
- [ ] Streak freezes, one earned every four weeks
- [ ] Achievement definitions and the unlock check
- [ ] Celebration screen for PRs and achievements
- [ ] Block completion unlocks the next program or a challenge

M10 · Runs, hikes, swims and steps

Done when: a weekend hike shows up on its own and pushes Monday's leg day back.

- [ ] Manual activity logging: free text parsed by the LLM, then confirmed
- [ ] Credit conditioning minutes to the week ledger and add activity fatigue to the planner (unit tests)
- [ ] Partial kcal credit for reported activity calories (unit tests)
- [ ] Apple Health sync for workouts and steps
- [ ] Android Health Connect sync
- [ ] Strava OAuth sync, de-duplicated by `externalId`

M11 · Meal planner

Done when: you can fit a favourite recipe to your targets and plan a week with a shopping list.

- [ ] `Recipe` schema and a create/edit recipe screen
- [ ] Per-serving kcal, protein and fibre from ingredient grams (unit tests)
- [ ] Recipe import: paste text, LLM turns it into ingredients and grams, you confirm
- [ ] Adjuster: solve ingredient grams within limits with javascript-lp-solver (unit tests)
- [ ] Lock ingredients and compare old vs new amounts
- [ ] Substitution table and swap suggestions when the solver can't hit targets
- [ ] Weekly meal plan screen with each day's projected totals

- [ ] Shopping list, and one-tap logging of a planned meal

M12 · Progress and adaptation

Done when: the app adjusts your calories from real results and shows you how far you've come.

- [ ] Weight trend chart with a 7-day moving average
- [ ] Adaptive TDEE after two weeks of logs (unit tests)
- [ ] Strength graph per exercise
- [ ] Monthly progress photos, side by side
- [ ] Baseline vs latest retest comparison

M13 · Polish and release

Done when: the app is on TestFlight and you'd trust a friend to use it.

- [ ] Exercise demo videos
- [ ] Error monitoring (e.g. Sentry) in app and API
- [ ] Delete account, data and photos
- [ ] App icon, splash screen, empty states
- [ ] TestFlight and Play internal testing builds via EAS